



FACULTAD DE INGENIERIA

INGENIERIA DE SISTEMAS COMPUTACIONALES

TECNICAS DE PROGRAMACION ORIENTADA A OBJETOS

TRABAJO DE CAMPO 4

“GESTION DE RECURSOS LOGISTICOS – JAVA”

NRC. 5076

ESTUDIANTE:

➤ Washington Jhoutman ALAYO JUAREZ (N00429999)

DOCENTE:

➤ Ing. Martin Eduardo TORRES RODRIGUEZ

2025 - 2 (IV CICLO)

Fecha de entrega: 27SET2025

TRUJILLO – PERU

2025

I. INTRODUCCION

1. PRESENTACION DEL PROYECTO:

El presente proyecto denominado “Gestión de Recursos Logísticos”, se desarrolla utilizando el lenguaje de programación JAVA, complementándose con la interfaz del edición de código VS Code, el cual propone el desarrollo de un aplicativo o consola, mediante técnicas de programación orientada a objetos, para gestionar inventarios, almacenes, pedidos, información de la descripción y características de un producto existente en una organización, resultado que busca optimizar el uso de recursos logísticos, reducir tiempos de respuesta, mejorar la trazabilidad y ofrecer interfaces flexibles para administración y reportes en tiempo real y a detalle, de los movimientos internos que se ejecutan en una empresa.

Asimismo, se busca establecer una solución adaptable a diferentes entornos organizacionales, que facilite la toma de decisiones estratégicas, promoviendo la automatización de procesos rutinarios, para minimizar los errores humanos y fortaleciendo la eficiencia operativa, convirtiéndose de esta manera, en una herramienta clave para la gestión integral de los recursos logísticos.

2. JUSTIFICACION:

En la actualidad, las empresas enfrentan desafíos en la coordinación de sus inventarios, esto debido a que muchas de ellas aun utilizan medios tradicionales para guardar el reporte y registro de entrada y salida, refiriéndonos al uso del lápiz y papel o aplicativos que no ofrecen garantía y son difíciles de entender, es por eso que, se presenta un sistema bien diseñado, que nos va a permitir reducir costos operativos, evitar rupturas de stock y mejorar la planificación logística.

3. OBJETIVOS GENERALES Y ESPECÍFICOS

Objetivo General: Diseñar e implementar un sistema orientado a objetos en Java para la gestión de recursos logísticos.

Objetivos Específicos:

- ✓ Crear clases (Producto, Almacén, Pedido, cliente).

- ✓ Desarrollar una interfaz gráfica (Swing/JavaFX).
- ✓ Mostrar sobrecargas, errores y relaciones
- ✓ Reducir tiempo y costo en la gestión logística.

4. ALCANCE

- Al desarrollar este sistema, se busca facilitar la gestión integral de recursos logísticos en una organización.
- Se propone brindar a los administradores y jefes de almacén una herramienta para registrar, consultar y controlar productos, pedidos y almacenes.
- Mejorar la eficiencia operativa, reduciendo errores manuales y optimizando el control de inventario.
- Apoyar a la toma de decisiones gerenciales mediante información confiable y centralizada en la base de datos.
- Beneficiar tanto a la empresa como a sus clientes finales, garantizando una logística más rápida, ordenada y transparente.

II. PLANTEAMIENTO DEL PROBLEMA:

¿Cómo gestionar eficientemente los recursos logísticos de una empresa para evitar pérdida de información, tiempo y costos, utilizando herramientas tecnológicas de programación orientada a objetos, que nos ayude a reconocer errores para el mejor control de los recursos?

2.1. Descripción

La falta de un sistema centralizado genera duplicidad de registros, errores en asignación de almacén y retrasos inevitables, además crea un presupuesto limitado por las malas gestiones, sumado a ello, tenemos infraestructura de red básica.

2.2. Restricciones y limitaciones realistas – Alternativas de Solución

Alcance funcional limitado (versión inicial del sistema):

- Actualmente el sistema cubre principalmente la gestión de productos, almacenes y pedidos en operaciones CRUD básicas.

- No contempla aún funciones más avanzadas como optimización de rutas logísticas, integración con sistemas ERP, ni gestión en tiempo real.
- Esto limita su uso a pequeñas y medianas empresas, donde la simplicidad es más prioritaria que la complejidad.

Base de datos embebida (H2):

- Se utilizó H2 como motor de base de datos en memoria o archivo local, lo cual es ideal para prototipos y pruebas.
- Restricción: en escenarios con gran volumen de datos o multiusuario en red, H2 no es la opción más robusta.
- Alternativa futura: migrar a un sistema más escalable (PostgreSQL, MySQL, SQL Server).

Dependencia de Java y JavaFX:

- El proyecto depende de que el usuario tenga instalado un JDK compatible (Java 17 o 21) y las librerías de JavaFX.
- ⊗ Restricción: limita la portabilidad inmediata en entornos donde no se usan tecnologías Java, a diferencia de soluciones web que corren en cualquier navegador.
- Alternativa: empaquetar el sistema como un JAR ejecutable con dependencias incluidas o migrar a una versión web.

Interfaz gráfica inicial (JavaFX):

- La GUI implementada en JavaFX cumple con funciones básicas (mostrar tablas, agregar/eliminar registros).
- ⊗ Restricción: carece de funcionalidades avanzadas como reportes gráficos, dashboards en tiempo real o integración con dispositivos móviles.
- Alternativa: ampliar la interfaz con más módulos o considerar frameworks multiplataforma/web para mayor alcance (ej. Angular + Spring Boot).

Seguridad de la información:

- La autenticación de usuarios y roles aún no está implementada, lo que limita el control de accesos en ambientes colaborativos.

- ⊗ Restricción: el sistema no protege contra accesos indebidos ni cuenta con encriptación de datos sensibles.
- Alternativa: agregar un módulo de gestión de usuarios y roles con autenticación (ej. JWT, OAuth, o sistema propio de login).

Escalabilidad y concurrencia:

- El sistema fue diseñado como prototipo académico y funciona bien en entornos de un solo usuario.
- ⊗ Restricción: no soporta de manera nativa múltiples usuarios concurrentes conectándose a la misma base de datos.
- Alternativa: migrar hacia una arquitectura cliente-servidor, con base de datos centralizada.

Limitación tecnológica (entorno local):

- El sistema funciona localmente en la PC con VS Code y Maven, lo que lo hace dependiente del entorno de desarrollo.
- ⊗ Restricción: no está aún preparado para despliegue en producción en la nube o servidores empresariales.
- Alternativa: empaquetar con Docker o desplegar en un servidor de aplicaciones como Tomcat o Spring Boot en la nube.

III. ANTECEDENTES

3.1. Revisión de Literatura y proyectos similares:

- **Trujillo Peralta, M. A. (2012). *Sistema de información en el proceso de gestión logística en el área de almacén de la empresa EGP Comunicaciones S.A.C. Tesis, Universidad César Vallejo, Perú.***

Desarrollaron un sistema de información para el área de almacén, con control de materiales, tiempos de entrega y fiabilidad en suministros.

- **Rojas C. E., & Vivas S. M. (2023). *Desarrollo de un sistema de gestión logística mediante la automatización de procesos (RPA) en la empresa RJ Electric. Tesis, UCV.***

Introdujeron automatización de tareas repetitivas en logística, (por ejemplo, generación automática de reportes, actualización de stock automática).

3.2. Referencias:

- ✓ Trujillo Peralta, M. A. (2012). *Sistema de información en el proceso de gestión logística en el área de almacén de la empresa EGP Comunicaciones S.A.C.* (Tesis de licenciatura). Universidad César Vallejo, Trujillo, Perú.
- ✓ Rojas Clemente, T. E., & Vivas Santiago, M. S. (2023). Desarrollo de un sistema de gestión logística mediante la automatización de procesos (RPA) en la empresa RJ Electric. (Tesis). UCV, Perú.

IV. REQUERIMIENTOS DEL SISTEMA:

4.1. Requerimientos funcionales:

- Registrar productos con atributos
- Eliminar productos.
- Listar productos con filtros por categoría.
- Editar almacenes.
- Registrar stock de productos por almacén.
- Crear pedidos (cliente, productos, cantidad, destino).
- Editar y eliminar pedidos.
- Registrar salida y llegada de productos/pedidos
- Generar reportes de registros.
- Gestión de prioridades en pedidos.
- Validaciones de entrada y control de errores.
- Interfaz interactiva en aplicación o entorno gráfico.
- Búsqueda avanzada por múltiples atributos.

4.2. Requerimientos no funcionales:

- ⊗ El sistema debe soportar al menos 50 usuarios concurrentes en la versión inicial.
- ⊗ Respuesta de consultas frecuentes.
- ⊗ Escalabilidad vertical inicial, diseño modular para escalar a microservicios si fuese necesario.
- ⊗ Compatibilidad con MySQL.

V. HISTORIAS DE USUARIO

5.1. Tablas con Historias de Usuario y Aceptación Esperada:

GESTION DE PRODUCTOS

ID	Historia de Usuario	Criterios de Aceptación	Estado
01	Como administrador logístico, quiero registrar productos con atributos como nombre y categoría, para tener un catálogo actualizado de inventario.	Al registrar, el sistema guarda el producto en la BD y asigna un ID único.	Implementado
02	Como administrador logístico, quiero listar todos los productos registrados, para ver de manera rápida el inventario actual.	Al ejecutar la opción de listar, se muestran todos los productos con sus atributos.	Implementado
03	Como administrador logístico, quiero eliminar productos que ya no se usan, para mantener limpio el sistema.	Al seleccionar un producto y eliminarlo, este desaparece de la lista.	Implementado
04	Como administrador logístico, quiero editar productos existentes, para actualizar su información.	Al actualizar un producto, los cambios se guardan y se reflejan en la lista.	Pendiente

GESTION DEL ALMACEN

ID	Historia de Usuario	Criterios de Aceptación	Estado
05	Como jefe de almacén, quiero crear registros de almacenes con nombre, ubicación y capacidad, para gestionar el espacio físico disponible.	Se puede crear un nuevo almacén y se almacena en la BD con un ID único.	Pendiente
06	Como jefe de almacén, quiero consultar la lista de almacenes existentes, para asignar productos y pedidos a un lugar específico.	Se listan todos los almacenes registrados con sus atributos.	Pendiente
07	Como jefe de almacén, quiero eliminar almacenes que ya no están en uso, para mantener la base de datos actualizada.	Al eliminar un almacén, este ya no aparece en el sistema.	Pendiente

GESTION DE PEDIDOS

ID	Historia de Usuario	Criterios de Aceptación	Estado
08	Como responsable de logística, quiero crear pedidos asignando productos y cantidades, para registrar solicitudes de clientes.	El pedido se guarda con su cliente, destino y productos asociados.	Pendiente
09	Como responsable de logística, quiero consultar los pedidos registrados, para dar seguimiento a su estado.	Al listar, se muestran los pedidos con su estado y productos asociados.	Pendiente
10	Como responsable de logística, quiero cambiar el estado de un pedido (pendiente, en tránsito, entregado), para controlar el flujo de entregas.	El sistema permite modificar el estado.	Pendiente
11	Como responsable de logística, quiero eliminar pedidos cancelados, para mantener el historial limpio.	El pedido eliminado deja de aparecer en las consultas.	Pendiente

INTERFAZ

ID	Historia de Usuario	Criterios de Aceptación	Estado
12	Como administrador logístico, quiero usar una interfaz de consola básica, para probar rápidamente funciones CRUD.	El sistema presenta un menú en consola con opciones de gestión.	Implementado
13	Como administrador logístico, quiero usar una interfaz gráfica JavaFX con tabla de productos, para gestionar el inventario de forma más intuitiva.	La GUI muestra los productos en tabla y permite agregar/eliminar.	Implementado
14	Como usuario, quiero que el sistema me muestre mensajes de error claros al ingresar datos inválidos, para corregirlos fácilmente.	Si se ingresa un dato incorrecto, aparece un mensaje de error en la GUI.	Implementado
15	Como usuario, quiero recibir confirmación al agregar o eliminar registros, para tener certeza de que la acción se realizó.	Tras agregar/eliminar, se muestra un mensaje de confirmación.	Implementado

VI. DISEÑO DEL SISTEMA

6.1. Diagramas UML:

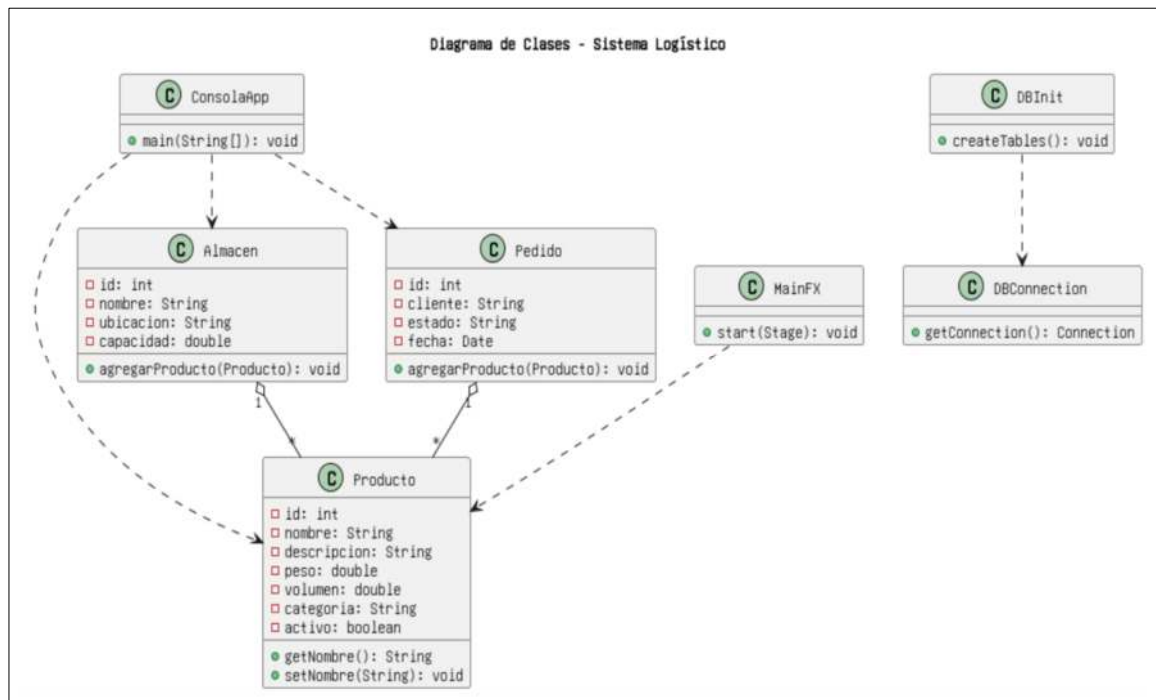
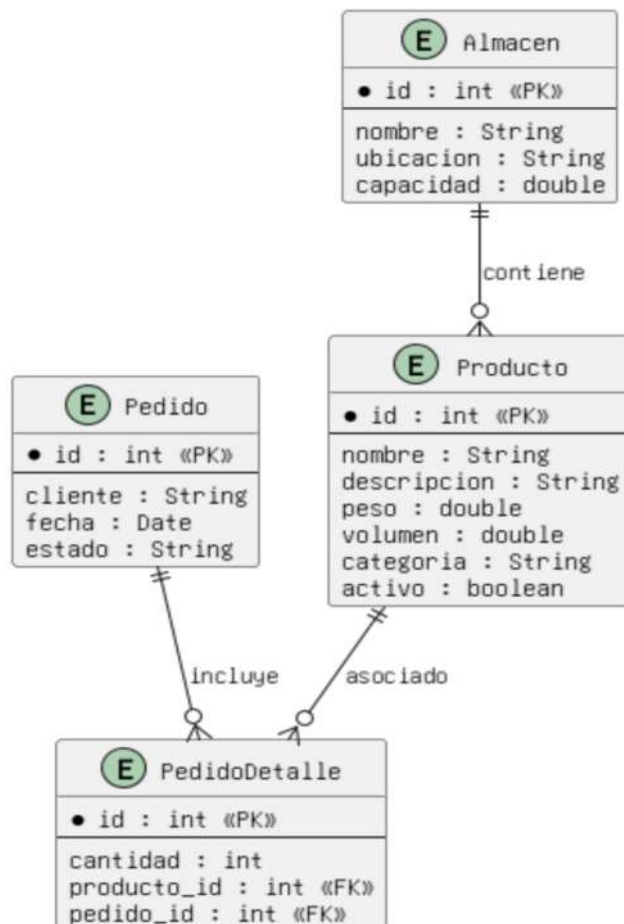


Diagrama de Relaciones - Sistema Logístico



6.2. Arquitectura propuesta:

Presentación (UI): Interacción con la consola.

- ConsolaApp: menú de opciones para CRUD básico.
- MainFX: interfaz gráfica para usuarios, con tablas, botones y mensajes.
- Objetivo: brindar diferentes formas de interacción (texto y gráfica).

Lógica de Negocio: ProductoService, PedidoService, AlmacenService:

- Contienen la lógica de negocio (validaciones, reglas de gestión).
- Se encargan de coordinar entre la UI y la base de datos.

Acceso a Datos: Ejecuta operaciones en la BD.

- DBConnection: establece la conexión con H2.
- DBInit: inicializa tablas y datos de prueba.
- (Opcional futuro: DAO específicos para cada entidad).
- Objetivo: encapsular toda la lógica de interacción con la base de datos.

Datos (Persistencia): Guarda la información en tablas.

- H2 Database: almacena productos, almacenes y pedidos.
- Tablas: Producto, Almacen, Pedido, PedidoDetalle.

6.3. Alcance de la solución:

El alcance de la solución es proveer una herramienta funcional, modular y accesible que cubra las principales operaciones de la gestión logística interna (productos, almacenes y pedidos), beneficiando tanto a administradores como responsables de logística, además, sienta las bases para evolucionar hacia un sistema más robusto en entornos empresariales reales.

VII. IMPLEMENTACION

7.1. Descripción del Desarrollo en JAVA:

El sistema sigue una arquitectura con la siguiente estructura de paquetes:

- `com.logistica.model`: Clases que representan las entidades principales: Producto, Almacen, Pedido.
- `com.logistica.service`: Contiene la lógica de negocio, con clases como `ProductoService`, que son las encargadas de validar y procesar operaciones.
- `com.logistica.db`: Maneja la conexión a la base de datos “`DBConnection`” y la inicialización de tablas “`DBInit`”.
- `com.logistica.ui`: incluye las interfaces de usuario:
 - `ConsolaApp`: (interfaz de consola).
 - `MainFX`: interfaz gráfica con JavaFX.

7.2. Funcionalidades Implementadas:

a) Modelado de Entidades:

Cada entidad del dominio logístico se implementó como una clase Java con atributos y métodos propios, para permitir que, que el sistema sea fácilmente extensible y mantenga la lógica organizada.

- `Producto`: representa artículos del inventario, con atributos como id, nombre, descripción, peso, volumen, categoría y activo.
- `Almacen`: representa espacios de almacenamiento con id, nombre, ubicación y capacidad.
- `Pedido`: agrupa productos solicitados por un cliente, con estado y fecha.

b) Conexión y Base de Datos:

Se utilizó H2 Database en modo archivo, lo que facilita las pruebas sin necesidad de instalar un gestor externo, donde cada ejecución asegura que la base de datos esté lista para usarse.

- `DBConnection`: implementa un método centralizado `getConnection()` para abrir conexiones JDBC.
- `DBInit`: ejecuta sentencias SQL de creación de tablas al iniciar la aplicación.

c) Lógica de Negocio (Servicios):

La capa de servicios conecta la interfaz con la base de datos:

- `ProductoService`: con los siguientes métodos `crearProducto`, `listarProductos`, `actualizarProducto`, `eliminarProducto`.

- Validaciones básicas (ejemplo: no registrar productos sin nombre).
- Manejo de excepciones para evitar que errores SQL afecten al usuario.

d) Interfaz de Usuario:

El sistema ofrece dos formas de interacción:

a. ConsolaApp :

- Menú con opciones numéricas para CRUD de productos, almacenes y pedidos.
- Facilita pruebas rápidas sin interfaz gráfica.

b. MainFX (JavaFX):

- Interfaz amigable con ventanas, tablas y botones.
- Permite visualizar el inventario, agregar productos y recibir mensajes de confirmación o error.
- Hace uso de TableView y PropertyValueFactory para mostrar datos de los productos.

e) Arquitectura Modular:

El desarrollo se organizó en capas independientes, para facilitar mantenimiento, pruebas unitarias y escalabilidad futura (migrar a MySQL o desplegar en web).

- Presentación: ConsolaApp, MainFX.
- Negocio: Servicios.
- Acceso a Datos: DBConnection, DBInit.
- Datos: Base de datos H2.

Herramientas Utilizadas:

- ❖ Java JDK 21: lenguaje de programación.
- ❖ JavaFX: interfaz gráfica.
- ❖ H2 Database: base de datos embebida.
- ❖ Maven: gestor de dependencias y compilación.
- ❖ Visual Studio Code: entorno de desarrollo.

7.3. Manejo de errores y colecciones aplicadas:

Manejo de Errores: Sirven para dar estabilidad y guiar al usuario:

- Uso de try-catch para capturar errores de conexión y SQL.
- Validación de entradas de usuario (no permitir datos vacíos o inválidos).
- Mensajes claros en consola y en GUI (JavaFX Alert).
- Evita operaciones inválidas.
- Confirmación de éxito después de cada operación CRUD.

Colecciones Aplicadas: para trabajar con datos dinámicos y mantener la interfaz actualizada automáticamente:

- ✓ ArrayList<Producto>: gestión de productos en memoria.
- ✓ ObservableList<Producto>: dinámica con tablas en JavaFX.
- ✓ List<Pedido> y List<Almacen>: manejo de múltiples entidades.

7.4. Códigos Relevantes:

Para insertar y listar productos dentro de ProductoServices.java:

```
public void crearProducto(Producto p)
{
    try (Connection conn = DBConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(
            "INSERT INTO producto(nombre, categoria) VALUES (?, ?)")) {
        ps.setString(1, p.getNombre());
        ps.setString(2, p.getCategoria());
        ps.executeUpdate();
    } catch (Exception e) { System.err.println("Error: " + e.getMessage()); }
}

public List<Producto> listarProductos()
{
    List<Producto> lista = new ArrayList<>();
    try (Connection conn = DBConnection.getConnection();
        ResultSet rs = conn.createStatement().executeQuery("SELECT * FROM
producto")) {
        while (rs.next()) lista.add(new Producto(rs.getInt("id"), rs.getString("nombre"),
rs.getString("categoria")));
    } catch (Exception e) { System.err.println("Error: " + e.getMessage()); }
    return lista;
}
```

Código que nos sirve para el diseño de interfaz de la ConsolaApp.java:

```
public static void main(String[] args) {
    DBInit.createTables();
    ProductoService ps = new ProductoService();
    ps.crearProducto(new Producto(0, "Laptop", "Tecnología"));
    ps.listarProductos().forEach(p ->System.out.println(p.getId()+"-"+ p.getNombre()));
}
```

Código para crear una tabla visual e interactiva en JavaFX.java:

```
public void start(Stage stage) {
    TableView<Producto> table = new TableView<>();
    table.setItems(FXCollections.observableArrayList(ps.listarProductos()));
    TableColumn<Producto, String> colNombre = new TableColumn<>("Nombre");
    colNombre.setCellValueFactory(c -> new
SimpleStringProperty(c.getValue().getNombre()));
    table.getColumns().add(colNombre);
    stage.setScene(new Scene(new VBox(table), 400, 300));
    stage.show();
}
```

Para garantizar la conexión con la base de datos H2:

```
public class DBConnection {
    private static final String URL = "jdbc:h2:./logisticaDB";
    public static Connection getConnection() throws SQLException {
        try { Class.forName("org.h2.Driver"); }
        catch (ClassNotFoundException e) { throw new SQLException("Driver H2 no
encontrado"); }
        return DriverManager.getConnection(URL, "sa", "");
    }
}
```

Para asegurar que la base de datos tenga las tablas necesarias:

```
public class DBInit {
    public static void createTables() {
        try (Connection conn = DBConnection.getConnection();
            Statement st = conn.createStatement()) {
            st.execute("CREATE TABLE IF NOT EXISTS producto (" +
                "id INT AUTO_INCREMENT PRIMARY KEY," +
                "nombre VARCHAR(200), categoria VARCHAR(100))");
        } catch (Exception e) { System.err.println("Error al crear tablas: " + e.getMessage()); }
    }
}
```

VIII. PROGRAMACION VISUAL Y ACCESO A DATOS

OPERACIONES CRUD IMPLEMENTADAS

Estas operaciones nos van a servir para garantizar que el sistema cubre las operaciones básicas de cualquier gestión de inventario.

- **Crear Producto:** Este método recibe un objeto Producto y lo guarda en la base de datos, evitándose errores con un mensaje de validación.
- **Consultar Producto:** Este método obtiene todos los productos de la tabla y los guarda en una lista de objetos Producto. Luego, esa lista se puede mostrar en la consola o en la interfaz gráfica.
- **Actualizar Producto:** Permite modificar el nombre de un producto existente identificado por su id. El sistema confirma si se actualizó con éxito o si no se encontró el registro.
- **Eliminar Producto:** Elimina un producto de la base de datos a partir de su id. El método devuelve mensajes claros para confirmar si la operación fue exitosa.

IX. ACTIVIDAD DE RESPONSABILIDAD SOCIAL

9.1. Descripción del proyecto realizado:

El proyecto desarrollado consistió en la implementación de un sistema de gestión de recursos logísticos, utilizando el lenguaje de programación Java bajo un enfoque de programación orientada a objetos, durante el desarrollo se aplicó una arquitectura en capas (presentación, lógica de negocio, acceso a datos y persistencia), lo que permitió mantener una estructura ordenada y escalable, además, se implementaron funcionalidades básicas de CRUD (crear, leer, actualizar y eliminar) sobre los principales elementos logísticos: productos, almacenes y pedidos, con validaciones y manejo de errores para garantizar confiabilidad, ofreciendo una propuesta de arquitectura que sustenta la solución de manera integral y fiable.

9.2. Impacto esperado:

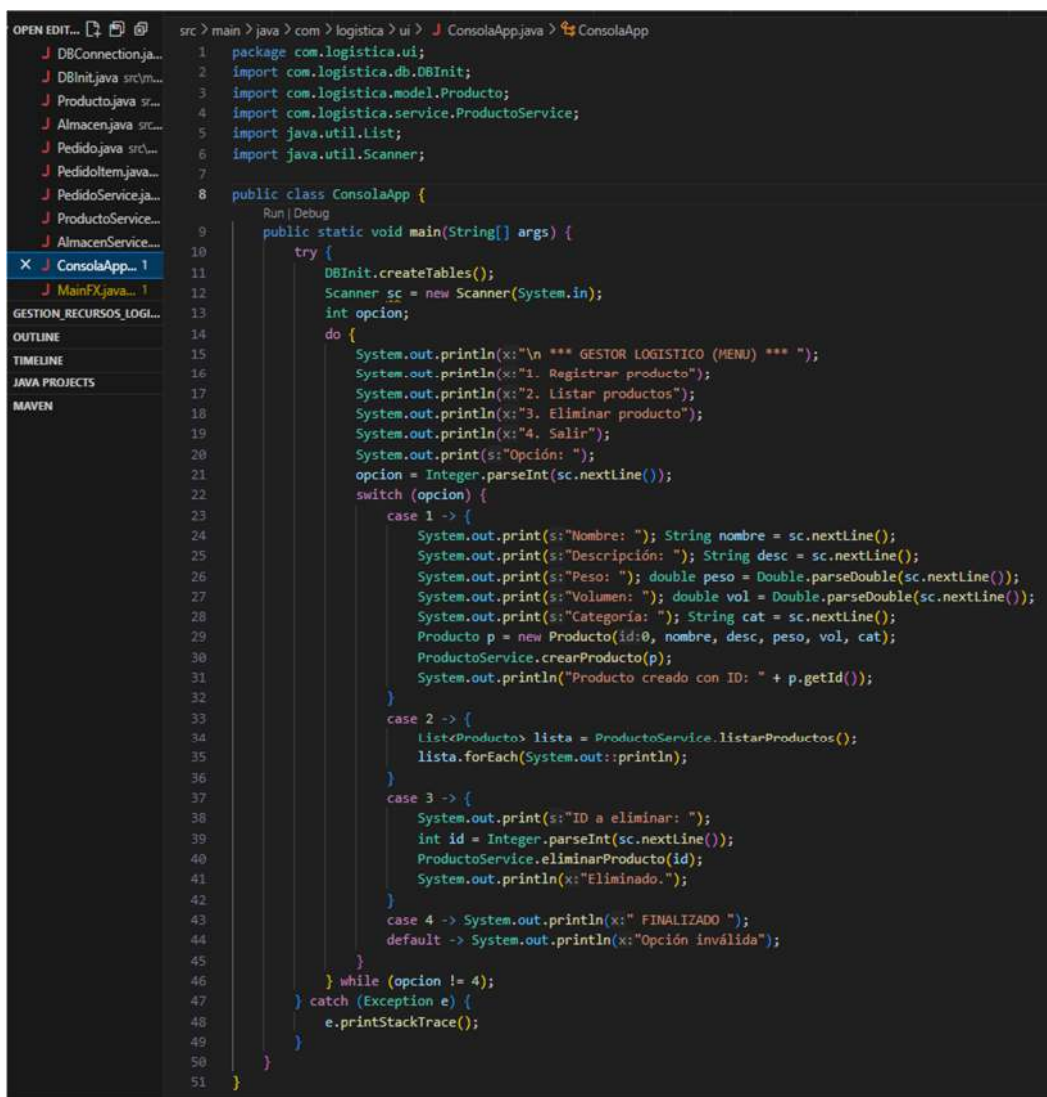
La implementación de este sistema busca generar un impacto positivo en la gestión logística mediante:

- ✓ Optimización de procesos internos: al digitalizar el registro, consulta y control de inventarios, almacenes y pedidos.
- ✓ Reducción de errores manuales: teniendo un manejo estructurado de datos y validaciones automáticas en el ingreso de información.
- ✓ Mejora en la toma de decisiones: al proporcionar información centralizada, confiable y accesible en tiempo real.
- ✓ Accesibilidad y usabilidad: mediante una interfaz sencilla, para pruebas rápidas y una interfaz gráfica (JavaFX) para un manejo más intuitivo.

X. RESULTADOS

10.1. Evidencia de ejecución del sistema:

Estructura del código de la clase “ConsolaApp”, se establece datos de entrada y salida, relaciones, manejo de errores y almacenamiento.



```

1  package com.logistica.ui;
2  import com.logistica.db.DBInit;
3  import com.logistica.model.Producto;
4  import com.logistica.service.ProductoService;
5  import java.util.List;
6  import java.util.Scanner;
7
8  public class ConsolaApp {
9      public static void main(String[] args) {
10         try {
11             DBInit.createTables();
12             Scanner sc = new Scanner(System.in);
13             int opcion;
14             do {
15                 System.out.println("\n *** GESTOR LOGISTICO (MENU) *** ");
16                 System.out.println("1. Registrar producto");
17                 System.out.println("2. Listar productos");
18                 System.out.println("3. Eliminar producto");
19                 System.out.println("4. Salir");
20                 System.out.print("Opción: ");
21                 opcion = Integer.parseInt(sc.nextLine());
22                 switch (opcion) {
23                     case 1 -> {
24                         System.out.print("Nombre: "); String nombre = sc.nextLine();
25                         System.out.print("Descripción: "); String desc = sc.nextLine();
26                         System.out.print("Peso: "); double peso = Double.parseDouble(sc.nextLine());
27                         System.out.print("Volumen: "); double vol = Double.parseDouble(sc.nextLine());
28                         System.out.print("Categoría: "); String cat = sc.nextLine();
29                         Producto p = new Producto(id:0, nombre, desc, peso, vol, cat);
30                         ProductoService.crearProducto(p);
31                         System.out.println("Producto creado con ID: " + p.getId());
32                     }
33                     case 2 -> {
34                         List<Producto> lista = ProductoService.listarProductos();
35                         lista.forEach(System.out::println);
36                     }
37                     case 3 -> {
38                         System.out.print("ID a eliminar: ");
39                         int id = Integer.parseInt(sc.nextLine());
40                         ProductoService.eliminarProducto(id);
41                         System.out.println("Eliminado.");
42                     }
43                     case 4 -> System.out.println(" FINALIZADO ");
44                     default -> System.out.println("Opción inválida");
45                 }
46             } while (opcion != 4);
47         } catch (Exception e) {
48             e.printStackTrace();
49         }
50     }
51 }

```


Estructura de la clase MainFx, para la compilación y ejecución posterior en la terminal local de la interfaz de VS Code, se utiliza línea de código: **mvn javafx:run**.

```

src > main > java > com > logistica > ui > MainFX.java > MainFX > start(Stage)
1  package com.logistica.ui;
2
3  import com.logistica.db.DBInit;
4  import com.logistica.model.Producto;
5  import com.logistica.service.ProductoService;
6  import javafx.application.Application;
7  import javafx.collections.FXCollections;
8  import javafx.collections.ObservableList;
9  import javafx.geometry.Insets;
10 import javafx.scene.Scene;
11 import javafx.scene.control.*;
12 import javafx.scene.control.cell.PropertyValueFactory;
13 import javafx.scene.layout.*;
14 import javafx.stage.Stage;
15 import java.util.List;

```

```

17 public class MainFX extends Application {
18     private TableView<Producto> table;
19     private ObservableList<Producto> data;
20     public static void main(String[] args) {
21         launch();
22     }
23     @Override
24     public void start(Stage stage) {
25         try {
26             DBInit.createTables();
27         } catch (Exception e) {
28             e.printStackTrace();
29         }
30         table = new TableView<>();
31         TableColumn<Producto, Integer> idCol = new TableColumn<>(text:"ID");
32         idCol.setCellValueFactory(new PropertyValueFactory<>(property:"id"));
33         TableColumn<Producto, String> nameCol = new TableColumn<>(text:"Nombre");
34         nameCol.setCellValueFactory(new PropertyValueFactory<>(property:"nombre"));
35         TableColumn<Producto, String> catCol = new TableColumn<>(text:"Categoría");
36         catCol.setCellValueFactory(new PropertyValueFactory<>(property:"categoria"));
37         TableColumn<Producto, Integer> descripcionCol = new TableColumn<>(text:"Descripción");
38         descripcionCol.setCellValueFactory(new PropertyValueFactory<>(property:"descripcion"));
39         table.getColumns().addAll(idCol, nameCol, catCol, descripcionCol);
40         refreshTable();
41         TextField tfNombre = new TextField();
42         tfNombre.setPromptText(value:"Nombre");
43         TextField tfDescripcion = new TextField();
44         tfDescripcion.setPromptText(value:"descripcion");
45         TextField tfCategoría = new TextField();
46         tfCategoría.setPromptText(value:"Categoría");
47

```

Se desarrolla la creación de botones funcionales para interactuar con la interfaz gráfica de JavaFX, además cuadros de texto donde se insertará o agregará contenido a la consola, para posteriormente mostrarse en un listado con ID, Nombre, Categoría y Descripción,

```
48 Button btnAgregar = new Button(text:" AGREGAR ");
49 btnAgregar.setOnAction(ev -> {
50     try {
51         Producto p = new Producto();
52         p.setNombre(tfNombre.getText());
53         p.setCategoria(tfCategoria.getText());
54         p.setDescripcion(tfDescripcion.getText());
55         p.setPeso(peso:0);
56         p.setVolumen(volumen:0);
57         ProductoService.crearProducto(p);
58         tfNombre.clear(); tfCategoria.clear();
59         refreshTable();
60     } catch (Exception ex) {
61         showAlert(title:"Error", ex.getMessage());
62     }
63 });
64 Button btnEliminar = new Button(text:" ELIMINAR ");
65 btnEliminar.setOnAction(ev -> {
66     Producto sel = table.getSelectionModel().getSelectedItem();
67     if (sel != null) {
68         try {
69             ProductoService.eliminarProducto(sel.getId());
70             refreshTable();
71         } catch (Exception ex) {
72             showAlert(title:" >>> Error <<< ", ex.getMessage());
73         }
74     }
75 });
76 HBox form = new HBox(8, tfNombre, tfCategoria,tfDescripcion, btnAgregar, btnEliminar);
77 form.setPadding(new Insets(10));
78 VBox root = new VBox(10, table, form);
79 root.setPadding(new Insets(10));
80 Scene scene = new Scene(root, 600, 400);
81 stage.setScene(scene);
82 stage.setTitle(" GESTION DE PRODUCTOS - JavaFX");
83 stage.show();
84
85 private void refreshTable() {
86     try {
87         List<Producto> list = ProductoService.listarProductos();
88         data = FXCollections.observableArrayList(list);
89         table.setItems(data);
90     } catch (Exception e) {
91         e.printStackTrace();
92     }
93 }
94 private void showAlert(String title, String msg) {
95     Alert a = new Alert(Alert.AlertType.ERROR, msg, ButtonType.OK);
96     a.setTitle(title);
97     a.showAndWait();
98 }
99 }
100
```

Botón funcional "ELIMINAR", para eliminar contenido seleccionado.

10.2. Pruebas realizadas:

10.2.1. COMPILACION DEL MAIN.JAVA:

El programa muestra el menú de opciones, en un listado del 1 al 4, a fin de elegir los requerimientos que solicite el usuario: Registrar, Listar o Eliminar un producto, además nos muestra una opción para finalizar consola, que se ejecuta cuando se culmina la tarea solicitada.

```
*** GESTOR LOGISTICO (MENU) ***
1. Registrar producto
2. Listar productos
3. Eliminar producto
4. Salir
Opción: 1
Nombre: IMPRESORA
Descripción: MULTIFUNCIONAL
Peso: 2.5
Volumen: 5
Categoría: ELECTRONICO
Producto creado con ID: 9
```

OPCION 01 (Registrar producto): ingresan diferentes características, detallando datos de nombre, descripción, peso, volumen, categoría, ID del producto.

```
*** GESTOR LOGISTICO (MENU) ***
1. Registrar producto
2. Listar productos
3. Eliminar producto
4. Salir
Opción: 2
Producto{id=1, nombre='impresora', categoria='electronicos', activo=true}
Producto{id=2, nombre='monitor', categoria='electronicos', activo=true}
Producto{id=3, nombre='teclado', categoria='electronicos', activo=true}
Producto{id=4, nombre='estabilizador', categoria='electronicos', activo=true}
Producto{id=5, nombre='escritorio', categoria='muebles', activo=true}
Producto{id=6, nombre='silla', categoria='muebles', activo=true}
Producto{id=7, nombre='estante', categoria='muebles', activo=true}
Producto{id=9, nombre='IMPRESORA', categoria='ELECTRONICO', activo=true}
```

OPCION 02 (Listar Producto): Al seleccionar esta opción se muestra el listado completo de los registros realizados en tiempo real y a detalle.

```
*** GESTOR LOGISTICO (MENU) ***
1. Registrar producto
2. Listar productos
3. Eliminar producto
4. Salir
Opción: 3
ID a eliminar: 9
Eliminado.
```

```
*** GESTOR LOGISTICO (MENU) ***
1. Registrar producto
2. Listar productos
3. Eliminar producto
4. Salir
Opción: 4
FINALIZADO
```

10.2.2. PRUEBAS REALIZADAS Y RESULTADO DE CONSOLA

PRODUCTO FINAL (INTERFAZ JAVAFX)

GESTION DE PRODUCTOS - JavaFX

ID	Nombre	Categoría	Descripción
Tabla sin contenido			

Nombre Categoría descripción AGREGAR ELIMINAR

INGRESO DEL CONTENIDO – BOTON AGREGAR

GESTION DE PRODUCTOS - JavaFX

ID	Nombre	Categoría	Descripción
14	monitor	electronicos	plasma

parlante electronico sonic AGREGAR ELIMINAR

LISTADO COMPLETO DE CONTENIDO AGREGADO

GESTION DE PRODUCTOS - JavaFX

ID	Nombre	Categoría	Descripción
14	monitor	electronicos	plasma
15	parlante	electronico	sonic
16	scaner	electronicos	inalambrico
17	camara web	electronico	portatil
18	escritorio	muebles	plegable
19	papelera	muebles	metal

Nombre Categoría descripción AGREGAR ELIMINAR

XI. CONCLUSIONES

11.1. Principales logros alcanzados:

- ✓ Desarrollo positivo de un sistema logístico básico que nos servirá como modelo para proyectos posteriores al trabajar con base de datos.
- ✓ Automatización de operaciones CRUD sobre productos, pedidos y almacenes, reduciendo errores manuales en la gestión.
- ✓ Desarrollo de interfaces complementarias como la consola para pruebas rápidas y JavaFX, para una experiencia de usuario más interactiva y visual.
- ✓ Documentación integral del proyecto con requerimientos, historias de usuario, restricciones y diagramas UML.

11.2. Retos enfrentados y cómo se resolvieron:

- ⊗ El desconocimiento de las aplicaciones que se utilizan tradicionalmente en el desarrollo de este tipo de programas, como Java y JavaFX, lo que causo enredos porque JavaFX no viene incluido en las versiones nuevas de Java y al intentar ejecutar, la ventana gráfica no aparecía.
- ❖ **Resolución:** Tutoriales en YouTube, descargar el SDK de JavaFX aparte y agregarlo en las configuraciones de ejecución (--module-path y --add-modules), para que las interfaces se ejecuten sin problemas.

- ⊗ No se encontraba como conectar con la base de datos (H2), ya que el mensaje de error siempre era “Driver no encontrado” o “No suitable driver” y esto pasaba porque el **JAR** del H2 no se encontraba en el classpath y no sabía cómo realizarlo.
- ✓ **Resolución:** Luego de varios intentos se supo que debía tenerse librería h2.jar dentro de la carpeta **LIB** del proyecto y luego agregarla al classpath o al pom.xml, ya que se usó el Maven.

- ⊗ Para poder estructurar el proyecto en capas, todo se contenía en un solo archivo y el código que se desarrollaba no tenía sentido, porque arrojaba muchos errores.

- ❖ **Resolución:** Fue dividir en paquetes: “db, model, service, ui”, para que este mas estructurado y resulte más fácil de trabajar.
- ⊗ Lograr un correcto Manejo de errores y validaciones, pues no se controló correctamente los errores y el programa, no mostro resultado esperado cuando el se ingresaba datos vacíos o números donde no se debía.
- ❖ **Resolución:** Se agregó validaciones antes de guardar datos y usar try-catch con mensajes comprensibles para guiar al usuario y se evite utilizar código extenso.
- ⊗ El tiempo y organización, fueron los peores enemigos en el desarrollo de este proyecto, dado que para lograr el resultado del proyecto requiere dedicación para poder trabajar con la documentación y orientación necesaria.
- ✓ **Resolución:** Se trabajo paso a paso, tratando de tener una idea clara de lo que se quería en la consola y luego poder ejecutarla en la interfaz gráfica y por último la elaboración del informe.

11.3. Impacto del proyecto en el aprendizaje de POO:

Más allá de la teoría, este proyecto permitió ver cómo la Programación Orientada a Objetos, no es solo un concepto abstracto, sino una herramienta poderosa para organizar y escalar soluciones en entornos reales como la logística, teniendo la idea que, su desarrollo permitió pasar de la teoría a su aplicación práctica, mostrando cómo los principios de objetos, encapsulamiento, colecciones y modularidad realmente ayudan a construir un sistema ordenado, funcional y escalable.

XII. BIBLIOGRAFIA

- Fernández, H. A. F. (2012). Programación orientada a objetos usando java. Ecoe Ediciones.
- Debrauwer, L., & Van der Heyde, F. (2016). UML 2.5: iniciación, ejemplos y ejercicios corregidos. Ediciones Eni.

- De Parga, C. J. (2021). UML. Arquitectura de aplicaciones en Java, C++ y Python. Ra-Ma Editorial.
- Pisco Mendoza, C. E. (2025). Sistema de gestión de almacén e inventario para disminuir los costos logísticos en la empresa Servicios Generales Wiñayq SRL.
- Rodriguez Avalos, R. J. (2022). Diseño de un sistema de almacenamiento e inventarios para reducir costos logísticos en una empresa importadora de accesorios vehiculares, Trujillo, 2022.
- <https://www.youtube.com/watch?v=aqCvtufXdso>
- <https://www.youtube.com/watch?v=VHy6xFXJ1Rw>
- <https://www.youtube.com/watch?v=B8tzOzyX8dg>

XIII. ANEXOS

https://github.com/Washington-del-cyber/C-Users-ASUS-Documents-gestion_recursos_logisticos

Link del repositorio GitHub